SWIGGY

SQL Insights & Strategic Analysis

Sophisticated SQL queries with intricate joins — uncovering customer, restaurant, and delivery-partner insights for strategic decision-making.

13 Business Questions • SQL Query • Result Screenshot

SUBMISSION · SQL CASE STUDY DECK

What this deck covers

Each of the following slides isolates one business question, the SQL query written to answer it, and a placeholder for the corresponding query-result screenshot.

Q1 – Q4

Customer Fundamentals

City filters, ratings, order participation, order counts per customer.

Q5 – Q9

Restaurant & Order Trends

Revenue by restaurant, top ratings, city-wise activity, recent orders.

Q10 – Q13

Delivery & Behavior Patterns

Delivery-partner performance, repeat-order behavior, cross-customer patterns.

01

QUESTION 1

Display all customers who live in 'Delhi'.

SQL QUERY

```
select customer_id,
name from customers
where city = "Delhi";
```

customer_id	name
2	Rohini Verma
5	Manish Kumar
18	Sonali Mishra

Result — Question 1

02

QUESTION 2

Find the average rating of all restaurants in 'Mumbai'.

SQL QUERY

```
select avg(rating)
from restaurants
where city ="Mumbai";
```

	avg(rating)
▶	4.300000

Result — Question 2

03

QUESTION 3

List all customers who have placed at least one order.

SQL QUERY

```
select c.customer_id
from customers c
where customer_id in
(select customer_id from orders o);
```

OR

```
select c.customer_id, c.name,
count(distinct o.order_id) as no_of_orders
from customers c
left join orders o
on o.customer_id = c.customer_id
where c.customer_id in
(select customer_id from orders o)
group by c.customer_id, c.name;
```

customer_id	name	no_of_orders
1	Amit Sharma	2
2	Rohini Verma	3
3	Rajesh Gupta	3
4	Sneha Mehta	2
5	Manish Kumar	4
6	Priya Singh	3
7	Vikas Reddy	3
8	Anjali Patel	3
9	Suresh Nair	1
10	Kavita Deshmukh	2
11	Vivek Bhatt	2
12	Meera Joshi	2
13	Pankaj Jain	2
14	Nidhi Saxena	3
15	Ashok Kumar	3

Result — Question 3

04

QUESTION 4

Display the total number of orders placed by each customer.

SQL QUERY

```
select o.customer_id, c.name, count(order_id)
from customers c
left join orders o
on o.customer_id = c.customer_id
group by o.customer_id, c.name;
```

customer_id	name	count(order_id)
1	Amit Sharma	2
2	Rohini Verma	3
3	Rajesh Gupta	3
4	Sneha Mehta	2
5	Manish Kumar	4
6	Priya Singh	3
7	Vikas Reddy	3
8	Anjali Patel	3
9	Suresh Nair	1
10	Kavita Deshmukh	2
11	Vivek Bhatt	2
12	Meera Joshi	2
13	Pankaj Jain	2
14	Nidhi Saxena	3
15	Ashok Kumar	3

Result — Question 4

05

QUESTION 5

Find the total revenue generated by each restaurant.

SQL QUERY

```
select r.restaurant_id, r.name,
coalesce(sum(o.total_amount),0) as total_revenue
from restaurants r
left join orders o
on o.restaurant_id = r.restaurant_id
group by r.restaurant_id, r.name order by r.name asc;
```

restaurant_id	name	total_revenue
10	Andhra Spice	4050.00
19	Awadhi Zaika	4150.00
24	Bengali Bhuri Bhoj	0.00
3	Biryani House	5300.00
20	Bombay Bhel	0.00
17	Chaat Junction	2150.00
7	Coastal Delight	2100.00
4	Curry Pot	3200.00
28	Fish Fry	0.00
12	Flavours of Bengal	4050.00
9	Gujarat Express	2550.00
16	Kerala Kitchen	950.00
30	Lucknowi Nawabi	0.00
18	Maharashtrian Ma...	2050.00
27	Marwari Bhojanalay	0.00
22	Paradise Biryani	0.00
11	Punjabi Tadka	900.00
15	Rajasthani Rasoi	2100.00
6	Royal Biryani	650.00
21	Saffron Spice	0.00
26	Shahi Dawat	0.00
13	South Treat	2950.00
1	Spice of India	1100.00
23	Tamil Treat	0.00
2	Tandoori Flames	1200.00
5	Taste of Punjab	600.00
29	Thane Tadka	0.00
14	The Great Indian ...	1600.00
25	Udupi Palace	0.00
8	Veggie Delight	1600.00

06

QUESTION 6

Find the top 5 restaurants with the highest average rating.

SQL QUERY

```
select r.restaurant_id , r.name, avg(f.rating) as avg_rating
from restaurants r
join orders o
on r.restaurant_id = o.restaurant_id
join feedback f
on o.order_id =f.order_id
group by r.restaurant_id , r.name
order by avg_rating desc limit 5;
```

restaurant_id	name	avg_rating
15	Rajasthani Rasoi	5.0000
17	Chaat Junction	5.0000
2	Tandoori Flames	5.0000
13	South Treat	5.0000
9	Gujarat Express	4.5000

Result — Question 6

07

QUESTION 7

Display all customers who have never placed an order.

SQL QUERY

```
select c.customer_id, c.name
from customers c
where customer_id not in
(select customer_id from orders o);
```

customer_id	name
24	Sonal Kaur
25	Vivek Malhotra
26	Divya Iyer
27	Rakesh Yadav
28	Mona Sharma
29	Sudha Pillai
30	Gaurav Khanna

Result — Question 7

08

QUESTION 8

Find the number of orders placed by each customer in 'Mumbai'.

SQL QUERY

```sql
select c.customer_id, c.name, c.city,
count(o.order_id) as number_of_orders_placed
from customers c
join orders o
on c.customer_id = o.customer_id
where c.city = "Mumbai"
group by c.customer_id, c.name, c.city;
```

customer_id	name	city	number_of_orders_placed
1	Amit Sharma	Mumbai	2
3	Rajesh Gupta	Mumbai	3
19	Arjun Desai	Mumbai	2
23	Ravi Singh	Mumbai	2

Result — Question 8

09

QUESTION 9

Display all orders placed in the last 30 days.

SQL QUERY

```sql
select order_id, date(order_date)
from orders
where order_date >= date_sub((select max(order_date) from orders),
interval 30 day);
```

order_id	date(order_date)
1	2024-08-01
2	2024-08-02
3	2024-08-04
4	2024-08-01
5	2024-08-03
6	2024-08-06
7	2024-08-03
8	2024-08-08
9	2024-08-02
10	2024-08-09
11	2024-08-01
12	2024-08-04
13	2024-08-05
14	2024-08-06
15	2024-08-10
16	2024-08-01
17	2024-08-11
18	2024-08-04

order_id	date(order_date)
18	2024-08-04
19	2024-08-06
20	2024-08-02
21	2024-08-09
22	2024-08-01
23	2024-08-05
24	2024-08-03
25	2024-08-08
26	2024-08-09
27	2024-08-10
28	2024-08-12
29	2024-08-03
30	2024-08-04
31	2024-08-11
32	2024-08-13
33	2024-08-14
34	2024-08-01
35	2024-08-02
36	2024-08-05

37	2024-08-07
38	2024-08-09
39	2024-08-02
40	2024-08-05
41	2024-08-08
42	2024-08-11
43	2024-08-07
44	2024-08-12
45	2024-08-03
46	2024-08-15
47	2024-08-04
48	2024-08-07
49	2024-08-10
50	2024-08-09
51	2024-08-13
52	2024-08-14
53	2024-08-01
54	2024-08-15

Result — Question 9

10

QUESTION 10

List all delivery partners who have completed more than 1 delivery.

SQL QUERY

```sql
select dp.partner_id, dp.name, COUNT(od.order_id) as total_deliveries
from deliverypartners dp
join orderdelivery od
on dp.partner_id = od.partner_id
group by dp.partner_id, dp.name
having COUNT(od.order_id) > 1 order by partner_id asc;
```

partner_id	name	total_deliveries
1	Amit Sharma	2
2	Ravi Kumar	5
3	Priya Patel	3
4	Suresh Reddy	6
5	Anita Desai	4
6	Rajesh Gupta	4
7	Sonia Agarwal	3
8	Vikram Singh	2
9	Sneha Iyer	2
12	Reena Rao	2
13	Mohit Saini	2
16	Ritika Sharma	2

Result — Question 10

11

QUESTION 11

Find the customers who have placed orders on exactly three different days.

SQL QUERY

```sql
select o.customer_id, c.name,
count(distinct order_date) as order_date_count
from orders o
join customers c
on o.customer_id = c.customer_id
group by customer_id
having order_date_count = 3;
```

customer_id	name	order_date_count
2	Rohini Verma	3
6	Priya Singh	3
8	Anjali Patel	3
14	Nidhi Saxena	3
15	Ashok Kumar	3
18	Sonali Mishra	3

Result — Question 11

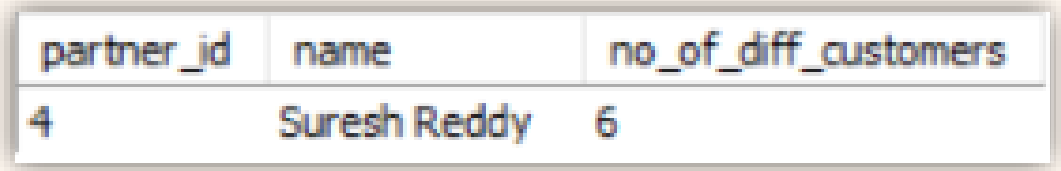
12

QUESTION 12

Find the delivery partner who has worked with the most different customers.

SQL QUERY

```
select od.partner_id, dp.name, count(distinct c.customer_id) as no_of_diff_customers from orderdelivery od
join orders o
on od.order_id = o.order_id
join customers c
on o.customer_id = c.customer_id
join deliverypartners dp
on od.partner_id = dp.partner_id
group by od.partner_id, dp.name
order by no_of_diff_customers desc limit 1;
```



partner_id	name	no_of_diff_customers
4	Suresh Reddy	6

Result — Question 12

13

QUESTION 13

Identify customers who have the same city and have placed orders at the same restaurants, but on different dates.

SQL QUERY

```
select c1.customer_id, c1.name, c1.city, o1.restaurant_id, date(o1.order_date),
c2.customer_id, c2.name, c2.city, o2.restaurant_id, date(o2.order_date)  from customers c1
join customers c2
on c1.city = c2.city                              -- same city
join orders o1
on c1.customer_id =o1.customer_id
join orders o2
on c2.customer_id = o2.customer_id
and o1.restaurant_id = o2.restaurant_id           -- same restaurant
and o1.order_date <> o2.order_date                -- different dates
and c1.customer_id < c2.customer_id;              -- non repeating customers
```

customer_id	name	city	restaurant_id	date(o1.order_date)	customer_id	name	city	restaurant_id	date(o2.order_date)
5	Manish Kumar	Delhi	3	2024-08-04	18	Sonali Mishra	Delhi	3	2024-08-05
19	Arjun Desai	Mumbai	8	2024-08-03	23	Ravi Singh	Mumbai	8	2024-08-09
5	Manish Kumar	Delhi	3	2024-08-07	18	Sonali Mishra	Delhi	3	2024-08-05

Result — Question 13

Thank You

13 queries answered, screenshots attached alongside each — a complete, join-driven view of Swiggy's customer, restaurant, and delivery data.

VINAY MEHRA

SWIGGY · SQL CASE STUDY SUBMISSION